



ResNet 논문 리뷰 — Deep Residual Learning for Image Recognition

논문 정보

- **제목:** Deep Residual Learning for Image Recognition
- **저자:** Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research)
- **발표:** CVPR 2016 (arXiv 2015.12) · ILSVRC 2015 Classification 우승
- **한 문장:** 아주 깊은 네트워크를 "잔차(residual) + shortcut" 구조로 재구성해, 깊이를 늘릴수록 성능이 오르게 만든 논문.
- **스터디:** CV 1주차 — CNN & Architectures · 원문 목차 순서대로 정리

Abstract

깊은 네트워크일수록 표현력은 커지지만 학습(최적화)은 오히려 더 어려워진다. 이 논문은 각 layer가 목표 매핑을 통째로 배우는 대신, **참조 입력(reference)에 대한 잔차(residual)를 학습**하도록 명시적으로 재구성하는 residual learning framework를 제안한다. 이렇게 하면 훨씬 깊은 네트워크도 최적화가 쉬워지고, 깊이에서 오는 정확도 이득을 실제로 얻을 수 있다.

ImageNet에서 최대 **152층**(VGG의 약 8배 깊이)까지 쌓았는데도 VGG보다 연산량이 낮고, 앙상블로 top-5 error 약 3.57%를 기록해 ILSVRC 2015 classification에서 1위를 했다. 또한 이 표현(representation)은 인식 문제 전반으로 일반화되어, ImageNet detection·localization과 COCO detection·segmentation까지 여러 대회 트랙을 함께 석권했다.

1. Introduction

핵심 질문: 네트워크를 깊게 쌓는 만큼 실제로 더 잘 배우는가?

깊이는 중요하다. VGG, GoogLeNet 등이 보여줬듯 layer를 더 쌓으면 저·중·고수준 feature가 풍부해지고 성능이 오른다. 그래서 자연스러운 질문이 생긴다 — "그냥 층을 더 쌓기만 하면 되는가?"

- **1차 장벽(gradient)**: vanishing / exploding gradient는 학습을 처음부터 방해했지만, normalized initialization(He init 등)과 중간 BatchNorm으로 수십 층 규모까지는 상당 부분 해결됐다. 즉 이게 유일한 문제가 아니다.
- **Degradation problem**: 네트워크가 깊어지면 정확도가 포화(saturate)된 뒤 **급격히 나빠진다**. 결정적으로, 이걸 test error만이 아니라 **training error도 함께 증가**한다. 훈련 오차까지 오른다는 건 **overfitting이 아니라 최적화 자체의 실패**라는 뜻이다. (논문 Fig.1: CIFAR에서 56층 plain이 20층 plain보다 train/test 둘 다 나쁨.)
- **구성적 해(constructed solution)로 본 모순**: 잘 학습된 얇은 모델을 가져와, 뒤에 붙는 추가 layer를 전부 identity mapping으로 두면 — 이 깊은 모델은 최소한 얇은 모델과 **같은** 오차를 내야 한다. 즉 "깊어서 못 배운다"는 건 원리적으로 말이 안 된다. 그런데 실제 SGD는 이 좋은 해(또는 그보다 나은 해)를 잘 못 찾는다. → solver가 여러 non-linear layer로 identity를 근사하는 걸 어려워한다는 신호.
- **제안(재구성)**: 그렇다면 layer가 근본 매핑 $H(x)$ 를 직접 배우게 하지 말고, **잔차** $F(x) := H(x) - x$ 를 배우게 하자. 원래 매핑은 $H(x) = F(x) + x$ 로 복원된다. identity가 최적이면 그냥 $F(x) \rightarrow 0$ 으로 weight를 죽이면 되는데, 이건 비선형 층으로 identity를 만드는 것보다 훨씬 쉽다.
- **구현**: 이 $+x$ 는 **shortcut connection**(층을 건너뛰어 입력을 그대로 더함)으로 구현한다. 파라미터도, 연산도 거의 늘지 않고, 기존 SGD/역전파를 그대로 쓸 수 있다.
- **결과 요약**: (1) 깊은 residual net은 최적화가 쉽고 plain net은 깊어질수록 train error가 오른다. (2) 깊이가 깊어질수록 정확도가 실제로 오른다. ImageNet과 CIFAR 양쪽에서 동일하게 확인된다.

2. Related Work

2.1 Residual Representations

문제를 "정답 그 자체"가 아니라 "기준점으로부터의 잔차"로 다루면 최적화가 쉬워진다는 건 딥러닝 이전부터 알려진 경험칙이다.

- 영상 인식의 **VLAD**는 사전(dictionary) 중심에 대한 잔차 벡터로 인코딩하고, **Fisher Vector**는 그 확률적 버전이다. 잔차를 인코딩하는 편이 원본을 직접 인코딩하는 것보다 강력하다.
- 수치해석의 **Multigrid**, 계층적 basis preconditioning도 문제를 여러 스케일의 잔차 하위문제로 쪼개어 수렴을 크게 빠르게 한다.
- 공통 교훈: 좋은 재구성(reformulation)·전처리(preconditioning)는 최적화를 단순하게 만든다. ResNet은 이 아이디어를 deep net에 가져온 것.

2.2 Shortcut Connections

층을 건너뛰는 연결 자체는 오래된 아이디어다.

- 초기 MLP에서 입력 ↔ 출력을 잇는 선형 skip, GoogLeNet의 보조 분류기(auxiliary classifier)처럼 중간 신호를 우회시키는 시도들이 있었다.
- 특히 **Highway Networks**(Srivastava et al., 2015)는 gating function으로 정보를 우회시킨다. 하지만 Highway의 gate는 학습되는 파라미터이고 데이터에 의존하며, gate가 닫히면(0에 가까워지면) 정보가 흐르지 않아 identity가 보장되지 않는다. 또 Highway는 극단적으로 깊어졌을 때 뚜렷한 정확도 이득을 보이지 못했다.
- **ResNet의 차별점**: shortcut이 파라미터 없는 순수 identity라 항상 열려 있고 절대 닫히지 않는다. layer는 오직 더해지는 잔차만 학습하면 된다. 그 결과 100층·1000층 규모에서도 안정적으로 학습되고 정확도가 오른다.

3. Deep Residual Learning

3.1 Residual Learning

원하는 이상적 매핑을 $H(x)$ 라 할 때, 기존 방식은 쌓인 층이 $H(x)$ 를 통째로 근사하게 한다. ResNet은 대신 층들이 잔차 함수를 근사하게 한다:

$$F(x) := H(x) - x \quad \Rightarrow \quad H(x) = F(x) + x$$

- **근사 능력은 같다:** 이론적으로 여러 비선형 층은 $H(x)$ 든 $F(x)$ 든 근사할 수 있다. 달라지는 건 **학습 난이도**다.
- **왜 더 쉬운가:** degradation 현상은 solver가 비선형 층 여러 개로 identity를 만드는 걸 어려워한다는 걸 시사한다. 잔차 방식에서는 최적해가 identity에 가까우면 그냥 $F(x)$ 의 weight를 0 근처로 보내면 되고, identity가 아니어도 이미 존재하는 x 를 **기준점**으로 두고 작은 보정만 배우면 되니 탐색 공간이 순해진다. 일종의 preconditioning.
- **실증:** 학습이 끝난 뒤 layer들의 잔차 응답(표준편차)을 재보면 plain 대비 **크기가 작다** (4.2). 즉 실제 최적해가 identity 근처에 있고, ResNet은 그 근처에서 작은 변화를 배우고 있다는 가설을 뒷받침.

3.2 Identity Mapping by Shortcuts

building block의 정의:

$$y = F(x, \{W_i\}) + x$$

- F 와 x 는 **element-wise 덧셈**이므로 **차원(shape)이 같아야** 한다. conv에서는 feature map 위에서 채널별로 더한다.
- 차원이 다를 때(채널↑, 해상도↓)만 선형 projection W_s 를 써서 맞춘다: $y = F(x, \{W_i\}) + W_s \cdot x$ (보통 1x1 conv, stride 2).
- **핵심:** W_s 는 오직 차원 맞춤용이다. 나머지 자리에서는 파라미터 없는 identity shortcut 만으로 충분하고, 이게 파라미터·연산을 안 늘려 경제적이며 plain과 공정 비교도 가능하게 한다.
- F 는 보통 **2~3개 층**. 층이 1개뿐이면 $y = W \cdot x + x$ 로 사실상 선형이 되어 이득이 없다(논문이 명시).

3.3 Network Architectures

Plain Network (VGG 철학 기반):

- 대부분 3x3 conv. 두 규칙 — (i) 같은 출력 해상도면 같은 채널 수, (ii) 해상도가 반으로 줄면 채널을 2배로 늘려 **layer당 연산량을 유지**.
- 다운샘플은 stride 2 conv로 처리, 끝에 global average pooling + 1000-way FC + softmax.

- 34층 plain의 연산량은 약 3.6 GFLOPs로, VGG-19(약 19.6 GFLOPs)의 **20% 이하**. 즉 훨씬 얇고 가볍다.

Residual Network:

- 위 plain에 shortcut을 더한다. 차원이 유지되는 구간은 identity, 차원이 바뀌는 경계에서만 두 옵션:
 - (A) zero-padding으로 채널을 늘려 identity 유지 (파라미터 0)
 - (B) 1×1 conv projection으로 맞춤
- basic block의 shape 흐름: $(B, C, H, W) \rightarrow \text{conv}3 \times 3 \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{conv}3 \times 3 \rightarrow \text{BN} \rightarrow \text{+x} \rightarrow \text{ReLU}$.

3.4 Implementation

- **입력 전처리**: 짧은 변을 [256, 480]에서 무작위 샘플(scale augmentation) 후 224×224 random crop, per-pixel mean 감산, horizontal flip, 표준 color augmentation.
- **정규화**: 각 conv 직후·activation 직전에 **BatchNorm** 적용. (dropout은 쓰지 않음.)
- **최적화**: SGD, mini-batch 256, 초기 lr 0.1에서 error가 정체하면 ÷10, weight decay 1e-4, momentum 0.9. 최대 60만 iteration 규모.
- **테스트**: 10-crop 평가, 그리고 여러 스케일에서 fully-convolutional 하게 점수를 낸 뒤 평균.

4. Experiments

4.1 ImageNet Classification

1000클래스 ImageNet 2012(학습 128만 장)에서 평가.

- **Plain Networks**: plain-34가 plain-18보다 **training error가 더 높다** → degradation을 그대로 재현. BN을 써서 forward 신호의 분산이 살아 있고 backward gradient도 정상 범위임을 확인했으므로, 이 열화는 vanishing gradient가 주원인이 아니라고 논문은 분석한다(수렴 속도가 지수적으로 느려지는 최적화 난이도로 추정).

- **Residual Networks:** 같은 구조에 shortcut만 더한 ResNet-34는 ResNet-18보다 낮은 error를 낸다 → degradation 해소. 게다가 (1) 깊이에서 오는 이득을 실제로 얻고, (2) 초기 수렴이 더 빠르다. 얇은 18층에서는 ResNet과 plain의 최종 정확도가 비슷하지만 ResNet이 더 빨리 수렴한다.
- **Identity vs. Projection Shortcuts:** 세 설정 비교 — (A) 차원 경계는 zero-pad(전부 파라미터 free) < (B) 차원 경계만 projection, 나머지 identity < (C) 모든 shortcut을 projection. 성능은 $A < B < C$ 지만 차이가 작다. C는 파라미터·메모리·연산을 크게 늘리므로, 논문은 **(B)를 기본으로 채택한다.** → projection은 degradation 해결의 필수 요소가 아니다(핵심은 residual 구조 자체).
- **Deeper Bottleneck:** 깊은 모델에서 연산을 아끼려는 설계. $1 \times 1(\text{채널} \downarrow) \rightarrow 3 \times 3 \rightarrow 1 \times 1(\text{채널} \uparrow)$ 3층 구조. 예: 입력 256 → 1×1 로 64 축소 → $3 \times 3(64)$ → 1×1 로 256 복원. 같은 시간 예산으로 훨씬 깊게 쌓을 수 있다. 여기서는 shortcut이 identity여야 양끝 고차원 텐서를 그대로 더할 수 있어 특히 중요하다(projection이면 비용이 2배로 커짐).
- **50/101/152-layer:** bottleneck으로 구성. 깊어질수록 정확도가 계속 오르고 degradation은 없다. ResNet-152는 VGG-16/19보다 깊지만 연산량(약 11.3 GFLOPs)은 더 낮다. 단일 모델 top-5 val error는 VGG류를 크게 앞선다.
- **Comparisons with SOTA:** 서로 다른 깊이의 ResNet 6개 앙상블로 ILSVRC 2015 classification 1위, top-5 test error 약 3.57% — 인간 수준에 근접한 당시 최고치.

모델	Block	Layer	비고
ResNet-18	Basic	18	baseline, 빠른 수렴 확인용
ResNet-34	Basic	34	plain-34와 직접 비교(degradation 증거)
ResNet-50	Bottleneck	50	실무 표준 backbone
ResNet-101	Bottleneck	101	detection/seg에서 자주 사용
ResNet-152	Bottleneck	152	ILSVRC 우승 모델, VGG보다 저연산

4.2 CIFAR-10 and Analysis

작은 데이터셋에서 깊이 자체의 효과를 통제된 조건으로 검증.

- **구조:** 32×32 입력, 첫 3×3 conv 뒤 $6n+2$ 층 형태. feature map은 {32, 16, 8}로 줄고 채널은 {16, 32, 64}로 늘며, 각 크기마다 $2n$ 개 층. $n = \{3, 5, 7, 9\} \rightarrow 20/32/44/56$ 층, 이어서 110층까지.

- **결과:** plain은 깊어질수록 악화되지만 ResNet은 20 → 110층으로 갈수록 계속 개선. 110층 ResNet이 당시 최고 수준의 낮은 error를 달성.
- **Analysis of Layer Responses:** 각 층 잔차 출력의 표준편차를 보면 ResNet이 plain보다 **작다**. 이는 "잔차는 0에 가깝고, 각 층은 대부분 작은 보정만 학습한다"는 3.1의 가설을 실증적으로 뒷받침한다. 또 깊을수록 개별 층이 신호를 덜 바꾼다.
- **Exploring Over 1000 Layers:** 1202층도 **최적화 자체는 문제없이** 되지만(train error가 매우 낮음), test 성능은 110층보다 오히려 살짝 나쁘다. 데이터에 비해 과한 용량 + 약한 정규화로 인한 overfitting으로 해석하며, 이는 residual 구조의 한계가 아니라 regularization 이슈다(maxout-dropout 등으로 개선 여지).

4.3 Object Detection on PASCAL and MS COCO

- 분류로 사전학습한 ResNet을 Faster R-CNN의 backbone으로 바꾸는 것만으로 PASCAL VOC 2007/2012, MS COCO에서 mAP가 크게 오른다.
- MS COCO의 표준 지표(mAP@[.5:.95])에서 VGG-16 baseline 대비 상대적으로 약 28% 향상. 엄격한 IoU 기준에서 이득이 크다는 건, 깊고 좋은 feature가 **정밀한 위치 추정**에 직접 기여한다는 의미.
- 결론적으로 ResNet의 표현은 분류에만 특화된 게 아니라 **downstream 인식 문제 전반으로 일반화**된다.

한 줄 요약

목표 함수 $H(x)$ 대신 잔차 $F(x) = H(x) - x$ 를 배우고 x 를 더하라. 파라미터 없는 identity shortcut으로 깊은 네트워크의 optimization을 가능하게 하고, 깊이의 이득을 실제 성능으로 바꾼 구조.